

# Test Description

## UML Diagrammer UI Test Suite

---

### Introduction

The purpose of the test suite is to test and verify the UML Diagrammer's basic functionality. The eventual goal is to keep extending the Diagrammer's capabilities while expanding the test suite to verify them. All the associated dependencies have unit tests. However, ad-hoc testing exposes defects because of code changes across modules or because of unexpected interactions between existing code and a new feature. The goal is to avoid expending an inordinate amount of time manually checking many of the basic features.

This test suite intends to provide a classic UI functional test for the UML Diagrammer and enable a high-quality tool.

Highly automated UI tests have always been the goal. However, finding a high-quality library that is scriptable, easy to maintain, and screen-resolution-independent is a challenge. The first versions of the automated test suite used [PyAutoGUI](#). This resulted in a [screen recording application](#) that aided in generating multiple UI functionality tests.

However, the screen recording application is tied to a specific display resolution when it captures screen locations, making the generated tests extremely brittle.

Eventually, the test suite took advantage of a PyAutoGUI feature that uses high-resolution screenshots. By comparing the screenshots to the UML Diagrammer, the test scripts know where to click and which dialogs to invoke. The tests use PyAutoGUI combined with the [opencv-python](#) project.

### Command-Line Invokable

One of the first requirements for the testing framework was that it should be invokable via a rational command-line interface. This satisfies the scriptable requirement. The goal is to run those tests via some CI (continuous integration) environment. The [Click](#) library provides this rational command-line interface.

## Testing Architecture

This project used an iterative approach to develop the suite. Thus, the initial implementation relied on screen-resolution-dependent coordinates. The current version takes the screen recognition tack.

To test various functionality, the test suite uses the “verifier” abstraction. A verifier tests a slice of functionality. For example, a large verifier would be one that can test and verify the creation of a UML Class. Other verifiers test whether the Diagrammer can create association relationships, interface relationships, etc.

The verifiers rely on “locators”. Locators are classes that can find UI objects on the UML Diagrammer application. For example, the `ClassDialogLocator` knows all about the dialogs associated with creating a class. A verifier relies on it to find the components that need to be “clicked” or interacted with to create projects that can be independently tested.

## PyAutoGUI Deficiencies

At least on macOS, PyAutoGUI fails because of timing difficulties and the fact that PyAutoGUI tries very hard to be a cross-platform testing scaffold.

On macOS, the test suite had difficulties with double-click actions and typing into various text input widgets. Research and consultation with the Gemini AI and Antigravity resulted in various dead ends. It took much consistent pressure and diligence to keep the AIs on point.

I isolated these deficiencies into a double-click handler and a typewrite handler. This sequestered the platform idiosyncrasies.

### MacOsDoubleClickHandler

This class, via its single class method (`doubleClick()`), dispatches a native macOS double-click event using Apple’s [Quartz Event Services](#). The method explicitly sets `kCGMouseEventClickState` to 1 for the first mouse-down/up pair and 2 for the second pair. This ensures that *macOS Cocoa* and *wxWidgets* properly register the gesture as a double-click.

### MacOsTypeWriteHandler

This class handles reliable text entry on *macOS* by routing keystrokes through AppleScript System Events.

PyAutoGUI’s built-in `typewrite()` simulates text entry by converting characters into raw hardware virtual keycodes and rapidly toggling synthetic Shift modifier keys for uppercase letters and special symbols.

On macOS, this approach is not reliable.